Software Requirements Specification (SRS)

Time Management Web App (TMWA)

Version 1.0 – November 2025

Prepared by:

- Sokmontrey Sythat (Project Manager / Developer)
- Ahmed Ashoor (Software Architect / Developer)
- Jonathan Cao (UI/UX Designer / Developer)
- Samuel Browne (QA Engineer / Developer)

Table of Content

Software Requirements Specification (SRS)

Time Management Web App (TMWA)

Version 1.0 – November 2025

Prepared by:

- Sokmontrey Sythat (Project Manager / Developer)
- Ahmed Ashoor (Software Architect / Developer)
- Jonathan Cao (UI/UX Designer / Developer)
- Samuel Browne (QA Engineer / Developer)

1.0 Introduction

1.1 Purpose

1.2 Scope

1.4 References

1.5 Overview of Document

2.0 Overall Description

2.1 System Environment

2.2 System Context

2.3 General Constraints

2.4 Assumptions and Dependencies

3.0 Functional Requirements

Feature #1 – Event & Task Management

Feature #2 – Availability Sharing

Feature #3 – Guest Booking

Feature #4 – Priority Recommendation

Feature #5 – Notification System

Feature #6 – Dashboard View

3.2 Use Cases

3.2.1: Manage Events and Tasks

3.2.2: Share Availability

3.2.3: Book Time via Shared Link

3.2.4: Task Priority Recommendation

3.2.5: Receive Notifications

3.2.6: View Dashboard

3.3 Data Modelling and Analysis

3.3.1 ER Diagram

3.3.2 Activity Diagram

3.3.3 Sequence Diagram

3.3.4 UML Diagram

3.4 Process Modelling

4.0 Non-functional Requirements

5.0 Logical Database Requirements

6.0 Approval

1.0 Introduction

1.1 Purpose

This document outlines the software requirements for the Time Management Web App (TMWA) our capstone project. It provides a clear picture of what the system aims to achieve, what features it includes, and how different parts of the application interact. The goal of this document is to help instructors, evaluators, and the development team share a common understanding of the system's purpose before moving into the design and implementation stages.

1.2 Scope

The Time Management Web App is a web-based productivity tool designed to help students, professionals, and small teams stay organized. It allows users to manage their schedules, plan tasks, and share their availability with others, making it easier to coordinate meetings and balance workloads collaboratively.

Key functions include:

- Task and event creation, modification, and tracking
- Priority-based task recommendations
- Recurring schedule management (daily, weekly, monthly views)
- Calendar sharing and common-time discovery between users
- Automatic reminders and notification system

Benefits:
TMWA reduces time conflicts, enhances collaboration, and simplifies planning via a user focused UI. The system focuses on efficiency and ease of navigation, addressing weaknesses observed in existing scheduling tools like Notion and Google Calendar.

In Scope

- Browser-based web interface
- Secure login / registration
- Calendar and availability management
- Task and event notifications

Out of Scope

- Native mobile / desktop / wearable apps
- AI-based natural-language scheduling
- Third-party calendar integration (e.g., Google Calendar API)

Term	Definition
User	Registered individual using the app to manage tasks and events.
Guest	Non-registered user accessing a shared availability link.
Event/Task	A scheduled activity with date, time, and optional priority.
Availability Link	A unique URL that allows others to view and book open time slots.
Priority Recommendation	An algorithmic suggestion based on deadlines / task type.
UI/UX	User Interface / User Experience; design focus areas for simplicity and flow.
Database	Central repository storing user accounts, events, and availability data.
Sprint	Iterative development cycle within the project timeline.

1.4 References

IEEE Std 830-1998 - IEEE Recommended Practice for Software Requirements Specifications

1.5 Overview of Document

Section 2 presents the overall system description - environment, user characteristics, and constraints.
Section 3 details functional and non-functional requirements, including use cases, data models, and security needs.

2.0 Overall Description

2.1 System Environment

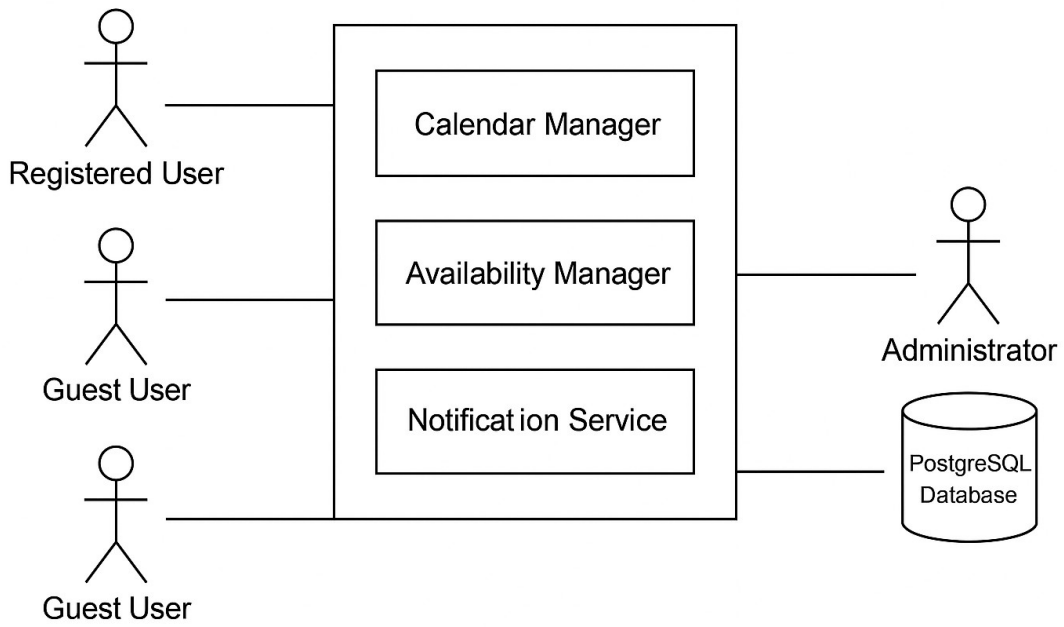


Figure 1 – System Environment

TMWA is structured like a three-tier web application, comprising a frontend built with HTML5 and CSS3 with JavaScript frameworks such as React or Next.js. The backend API is powered by either Node.js or Spring Boot using Java 21, delivering RESTful services. Data is managed through a PostgreSQL database accessed via Spring Data JPA. Our application is hosted through cloud platform AWS. Users engage with the system through modern browsers such as Chrome, Edge, or Firefox, whilst communication between the frontend and backend occurs over HTTPS REST APIs.

2.2 System Context

The system serves three user roles:

Actor	Description
Registered User	Creates tasks, manages schedules, shares availability.
Guest User	Views shared availability links and books time slots.
Administrator (Internal)	Maintains database and monitors system health.

External Dependencies

- Web server / database hosting service
- Internet connection
- Browser support for ES6 + JavaScript

2.3 General Constraints

The application must function within a browser environment such as Chrome, Edge, or Firefox and requires active internet connectivity. It is scheduled to be built and delivered between September 2025 and March 2026. The development will utilize open-source tools that adhere to their respective license terms, and the backend will be powered by a PostgreSQL database hosted on a cloud service provider like AWS.

2.4 Assumptions and Dependencies

We’re already assuming that users know how to handle basic scheduling and task management. Most people will be using the web system on a smart device and have a stable internet connection while doing so. To keep everything running smoothly, our system depends on cloud services for hosting and database access. The team will be using GitHub to manage code and Discord to stay in touch if anything goes wrong. The project will be built using Spring Boot for the backend and React for the frontend, with Java 21 powering the backend API.

3.0 Functional Requirements

Feature #1 – Event & Task Management

Introduction:
Allows users to create, edit, and delete events and tasks with optional priorities, deadlines, and reminders.

Inputs:

- Event title, date, time, description, and priority level
- User authentication token

Processing:

- Validates inputs and stores event details in database
- Automatically calculates event priority

Outputs:

- Updated event list displayed in calendar view
- Notifications generated for upcoming events

Feature #2 – Availability Sharing

Introduction:
Enables users to mark time slots as available/unavailable and generate shareable links for others.

Inputs:

- Selected time ranges
- Sharing permissions

Processing:

- Generates availability link with unique token
- Updates calendar records and database

Outputs:

- Public link accessible by guests
- Shared view of availability calendar

Feature #3 – Guest Booking

Introduction:
Allows guests to book a meeting or slot with a registered user via a shared link.

Inputs:

- Guest name, email, selected time slot

Processing:

- Verifies slot availability and reserves booking
- Updates both user and guest calendars

Outputs:

- Confirmation notification sent to both parties

Feature #4 – Priority Recommendation

Introduction:
Provides recommendations on which tasks to complete first based on user workload, deadlines, and preferences.

Inputs:

- User's current task list and due dates

Processing:

- Applies priority algorithm to rank tasks
  Adjusts rankings dynamically as new tasks are added

Outputs:

- Updated task list with "recommended next task" indicator

Feature #5 – Notification System

Introduction:
Notifies users about upcoming events, approaching deadlines, and new bookings.

Inputs:

- Event timestamps, notification settings

Processing:

- Background scheduler checks for time-based triggers
- Sends email / in-app notification

Outputs:

- Email alert or pop-up reminder

Feature #6 – Dashboard View

Introduction:
Centralized interface displaying all user events, reminders, and upcoming activities.

Inputs:

- User login credentials

Processing:

- Fetches all event and task data from database
- Compiles summary statistics

Outputs:

- Graphical dashboard with progress charts and task summaries

3.2 Use Cases

3.2.1: Manage Events and Tasks

Actors: Registered User
Description: Users can create, edit, delete, and view events and tasks with optional reminders and priorities.
Preconditions:

- Users must be logged in.
- Database connection is active.

Main Flow:

1. The user navigates to the “Tasks/Events” page.
2. The user clicks “Create New Task/Event.”
3. System prompts for title, description, date, time, and priority.
4. The user fills in the details and submits.
5. The system validates data and saves it to the database.
6. The updated task list is shown in the calendar view.

Alternative Flows:

- If validation fails, the system prompts for corrections.
- Users can edit or delete an existing task instead of creating a new one.

Postconditions:

- The task or event is stored and displayed in the user’s dashboard.
- Notifications are scheduled if reminders are set.

3.2.2: Share Availability

Actors: Registered User
Description: Users mark time slots as available or unavailable and share them with others.
Preconditions:

- The user is authenticated.

Main Flow:

1. The user navigates to the “Availability” section.
2. The user selects time slots to mark as available.
3. The user clicks “Generate Shareable Link.”
4. The system creates a unique availability URL with a secure token.
5. The user shares the link externally.

Alternative Flows:

- Users cancel before saving availability.
- The user modifies availability times later.

Postconditions:

- Availability data is updated in the database.
- Link is active for guest access.

3.2.3: Book Time via Shared Link

Actors: Guest User, Registered User
Description: A guest books a meeting slot with a registered user via the shared link.
Preconditions:

- A valid availability link exists.

Main Flow:

1. Guest opens the shared link.
2. Available time slots are displayed.
3. The guest selects a slot and enters name and email.
4. The system checks slot availability.
5. Booking is confirmed and both parties are notified.

Alternative Flows:

- If the slot is already taken, the system prompts to choose another.

Postconditions:

- Slot is reserved in both calendars.
- Notification sent to user and guest.

3.2.4: Task Priority Recommendation

Actors: Registered User
Description: System suggests which tasks should be done first based on deadlines and workload.
Preconditions:

- The user has existing tasks with due dates.

Main Flow:

1. The user accesses the dashboard.
2. The system analyzes task lists using the priority algorithm.
3. Tasks are ranked by urgency and importance.
4. “Recommended Next Task” is displayed.

Alternative Flows:

- The user manually adjusts task priorities.

Postconditions:

- Updated task list with ranked priorities visible to the user.

3.2.5: Receive Notifications

Actors: Registered User
Description: The system alerts users of upcoming events, deadlines, or new bookings.
Preconditions:

- Users have scheduled tasks or events.
- Notification settings are configured.

Main Flow:

1. The background scheduler monitors time-based triggers.
2. When an event is approaching, a notification is sent via email or in-app alert.
3. The user acknowledges or dismisses the notification.

Alternative Flows:

- Users disable certain types of notifications.

Postconditions:

- Users are reminded before events or bookings occur.

3.2.6: View Dashboard

Actors: Registered User
Description: The user can view all upcoming events, progress summaries, and tasks in one interface.
Preconditions:

- The user is logged in.

Main Flow:

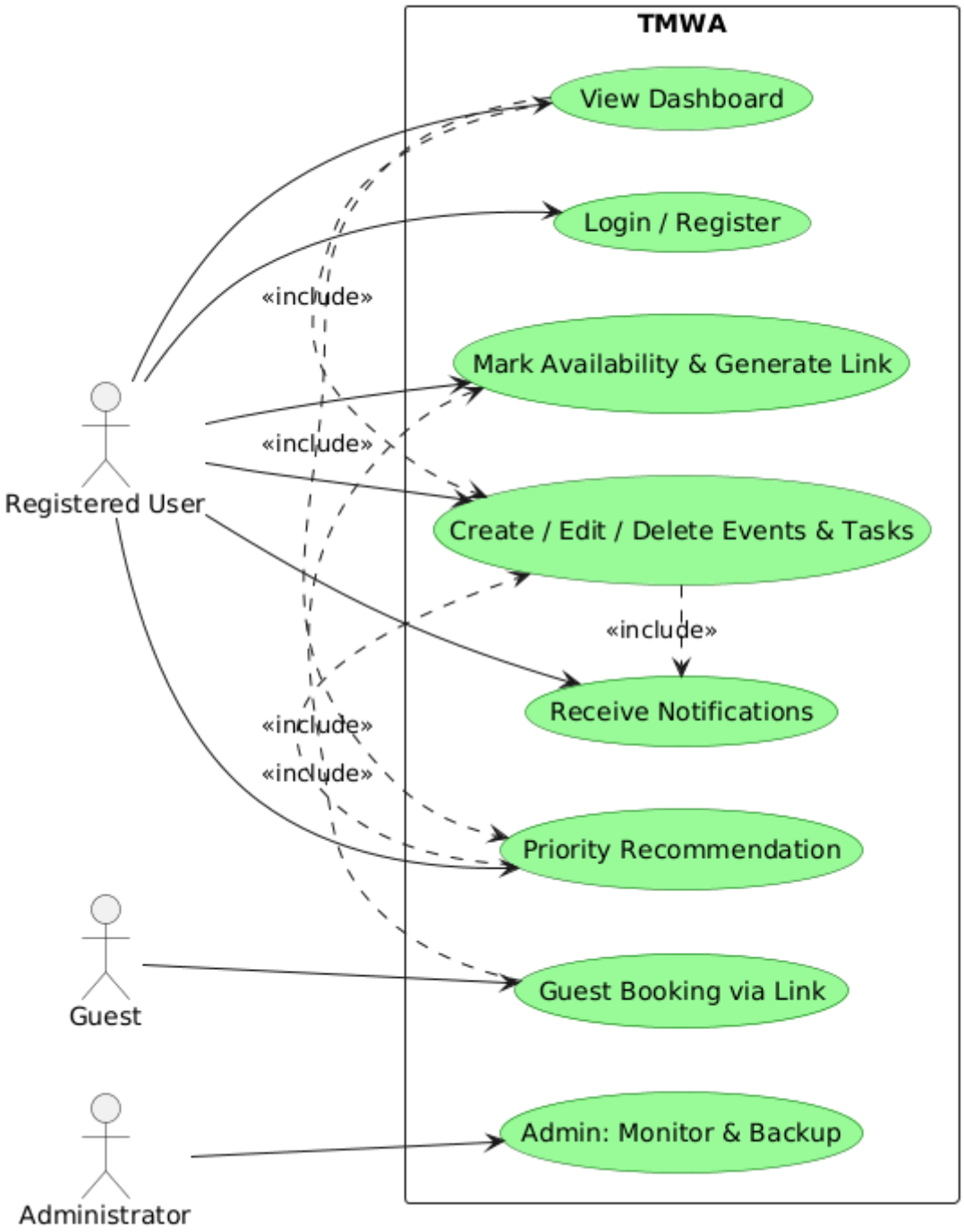
1. The user opens the dashboard.
2. The system retrieves tasks, events, and reminders from the database.
3. Dashboard displays graphical summaries and progress stats.

Alternative Flows:

- User filters or sorts data by date, category, or priority.

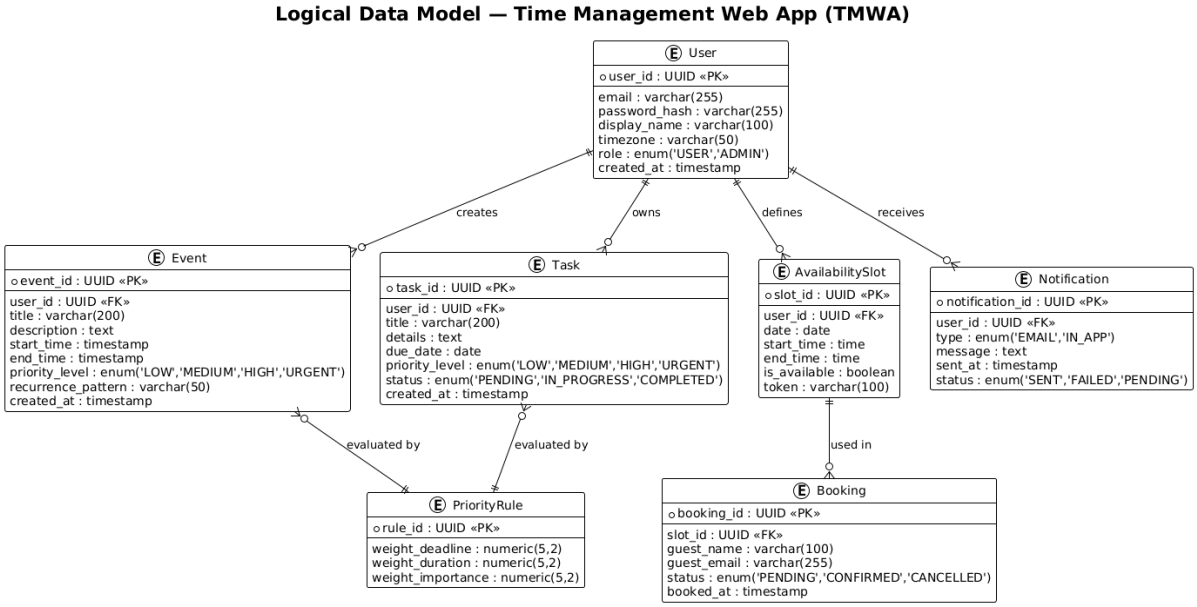
Postconditions:

- The user sees an up-to-date overview of schedule and performance.

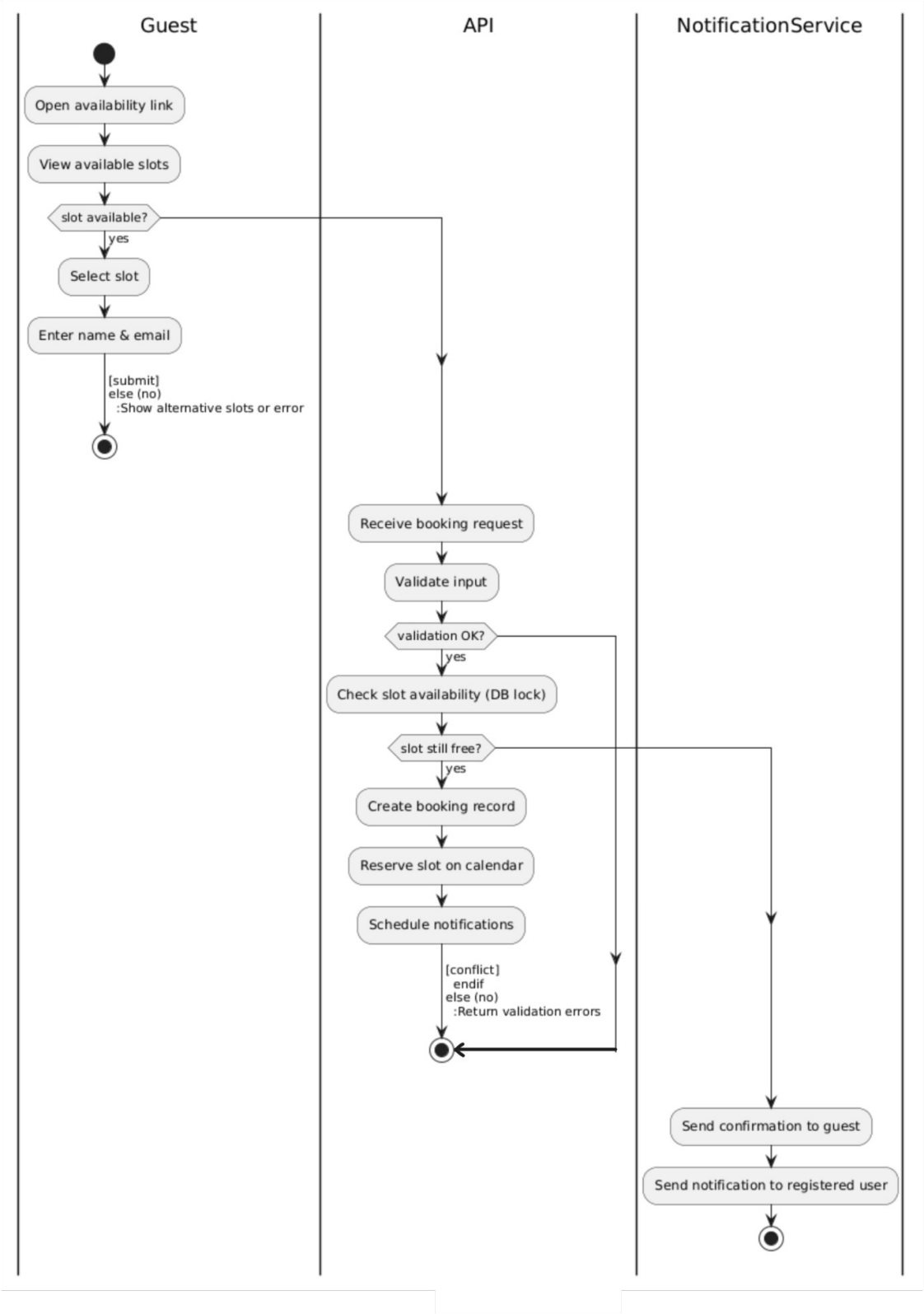


3.3 Data Modelling and Analysis

3.3.1 ER Diagram

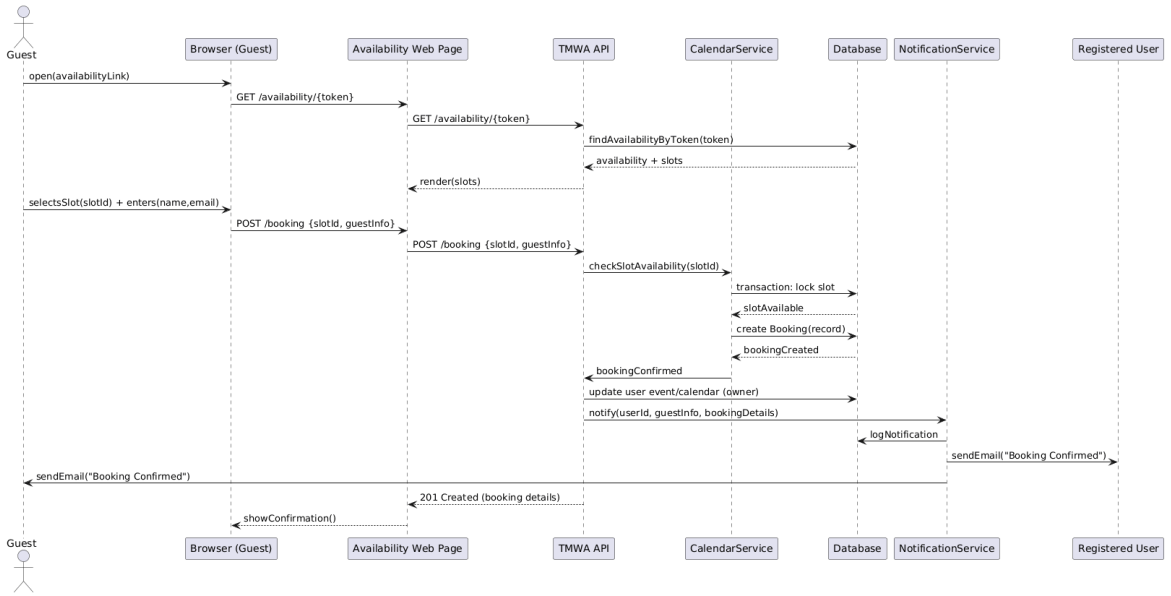


3.3.2 Activity Diagram

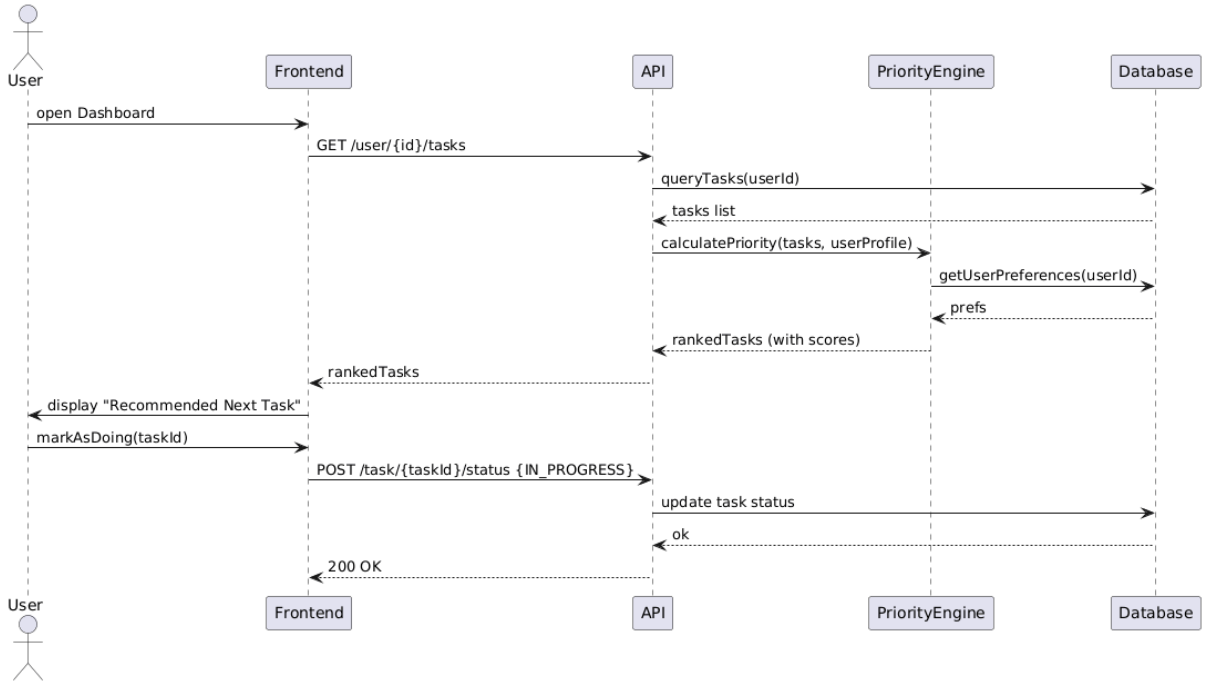


3.3.3 Sequence Diagram

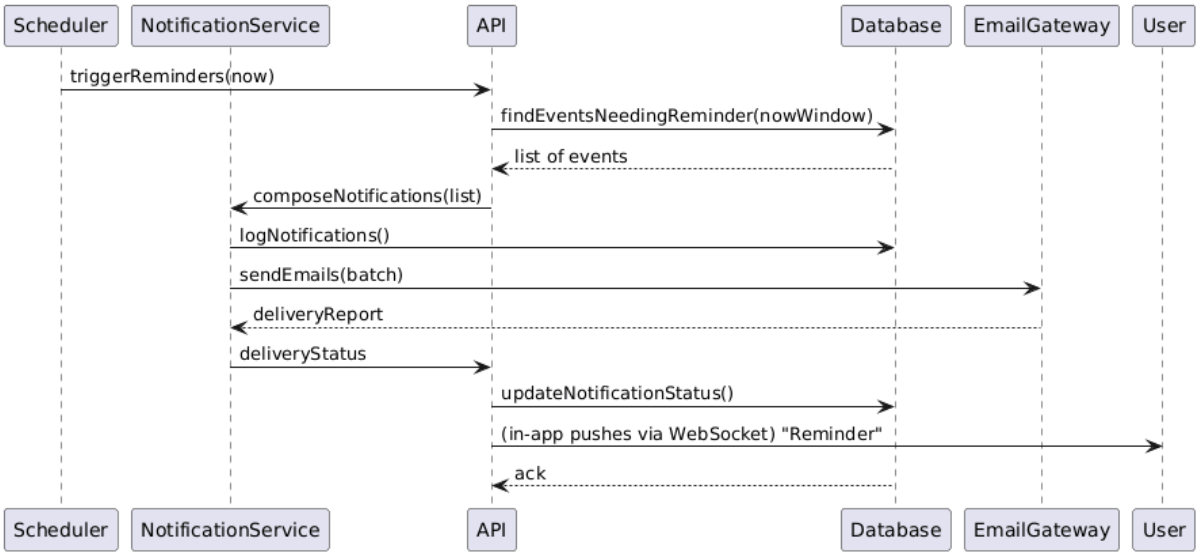
Guest Booking via Availability Link



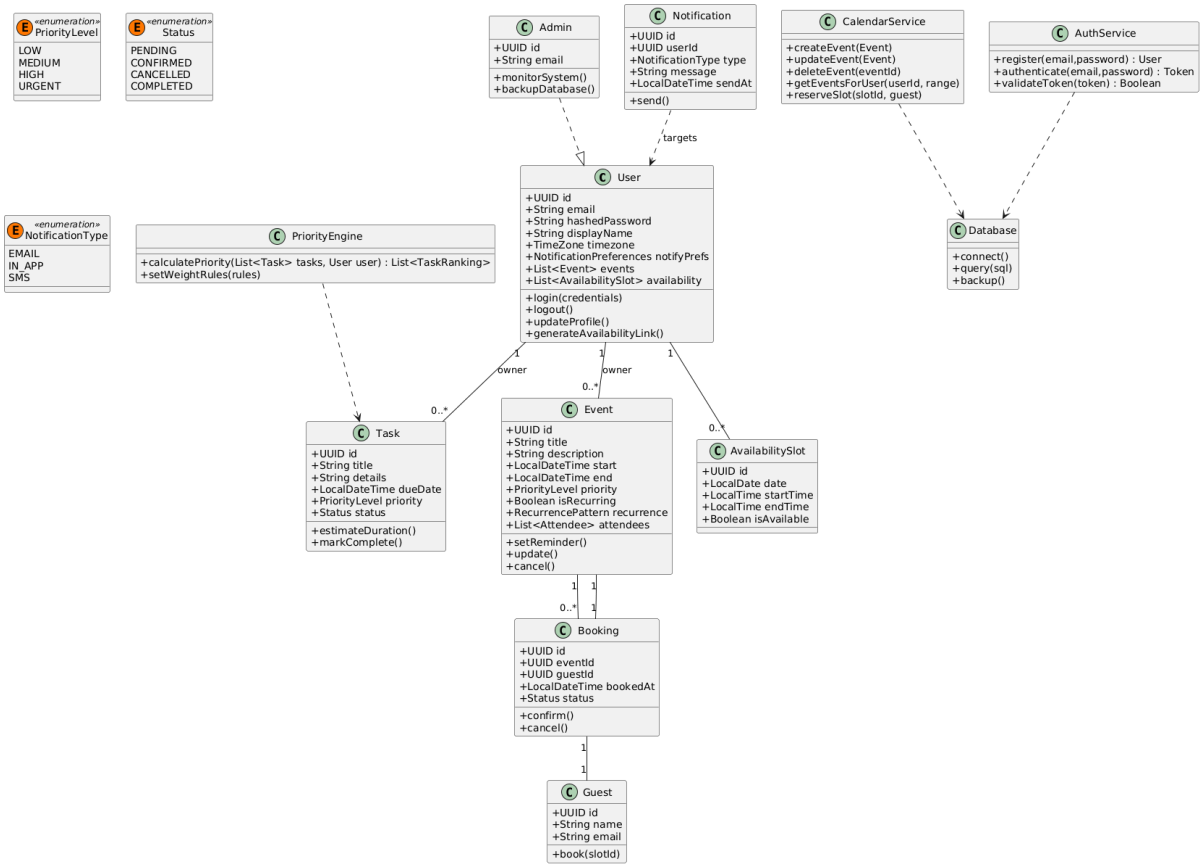
Priority Recommendation Flow



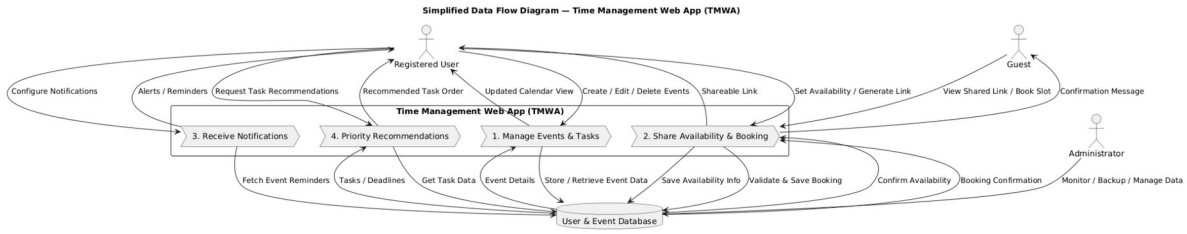
Notification Delivery (Reminder)


Scheduler
NotificationService
API
Database
EmailGateway
User
triggerReminders(now)
findEventsNeedingReminder(nowWindow)
list of events
composeNotifications(list)
logNotifications()
sendEmails(batch)
deliveryReport
deliveryStatus
updateNotificationStatus()
(in-app pushes via WebSocket) "Reminder"
ack
Scheduler
NotificationService
API
Database
EmailGateway
User

3.3.4 UML Diagram



3.4 Process Modelling



4.0 Non-functional Requirements

Category	Requirement	Criteria
Performance	App loads in under 2 seconds	95% of requests
Reliability	99% uptime during testing phase	Verified via monitoring
Security	All data encrypted via HTTPS	End-to-end
Maintainability	Modular MVC design	Independent components
Portability	Browser-accessible	Cross-platform
Usability	Minimal learning curve	Target: ≤10 min onboarding

The Time Management Web App (TMWA) prioritizes on performance, security, and a friendly user interface to ensure a smooth experience for the user. The system loads within two seconds for most user actions under normal conditions. It must remain reliable, with a target of 99% uptime during operation.

User data must be secured through our HTTPS encryption, hashed passwords, and role-based authentication. Privacy and safe data handling is important for protecting its users and meeting institutional standards.

The interface should be easy enough for new users to learn within minutes, minimizing complexity. The system should follow modular architecture principles for easy maintenance and updates. It will be easily accessible across browsers, ensuring cross-platform compatibility without requiring any installation.

5.0 Logical Database Requirements

The Time Management Web App (TMWA) stores and manages critical information such as user accounts, events, availability schedules, and bookings in a PostgreSQL relational database. To ensure accuracy, consistency, and less data duplication, the database is structured and normalized to third normal form (3NF).

Strict authentication procedures restrict database access, and sensitive data including user passwords is safely secured. By hosting the database on a cloud platform, information is always accessible and the structure can readily grow as more people join. To guard against data loss or unforeseen problems, regular automated backups are planned.

One-to-many links between users, events, and reservations are supported by the database design. The program can easily obtain, update, and synchronize thanks to this structure.

6.0 Approval

The signatures below indicate their approval of the contents of this document.

Project Role	Name	Signature	Date
Project Manager / Developer	Sokmontrey Sythat	S.S	2025-11-07
Software Architect / Developer	Ahmed Ashoor	A.A	2025-11-07
UI/UX Designer / Developer	Jonathan Cao	J.C	2025-11-07
QA Engineer / Developer	Samuel Browne	S.B	2025-11-07